Delays in Verilog HDL

Delay Modeling and Specify Blocks

October 12, 2026

Contents

1 Introduction **3**

2 Timescale Directive **3**
2.1 Syntax . . . 3
2.2 Example . . . 3

3 Types of Delays in Verilog **4**

4 Continuous Assignment Delay **4**
4.1 Syntax . . . 4
4.2 Example . . . 4

5 Gate-Level Delay **5**
5.1 Example: AND Gate . . . 5
5.2 Separate Rise, Fall, and Turn-Off Delays . . . 5

6 Procedural Delay **6**
6.1 Example . . . 6

7 Intra-Assignment Delay **6**
7.1 Example . . . 6

8 Min, Typical, and Max Delays **7**
8.1 Syntax . . . 7
8.2 Example . . . 7

9 Specify Block **8**
9.1 General Syntax . . . 8

10 Parallel and Full Module Paths **8**
10.1 Parallel Path . . . 8
10.2 Full Path . . . 9

11 Complete Specify Block Example **9**

12 Three-Value Path Delays **10**
12.1 Example . . . 10

13 Edge-Sensitive Module Paths **10**
13.1 Example 10

14 Conditional Specify Paths **11**
14.1 Example 11

15 Specify Block Parameters **12**
15.1 Example 12

16 Timing Checks in Specify Blocks **12**

17 Example: Setup and Hold Timing Checks **13**

18 Example: Clock Period and Pulse Width **14**

19 Specify Block vs. Assignment Delay **14**

20 Important Notes About Specify Blocks **15**

21 Common Errors and Best Practices **15**

22 Summary **16**

1 Introduction

Delay modeling in Verilog HDL describes the time required for signals to change during simulation.

In practical digital circuits, gates and interconnections introduce propagation delays. Verilog allows designers to represent these delays in simulation models.

Delay modeling is useful for understanding signal timing, propagation behavior, and timing relationships between circuit signals.

The main delay modeling constructs are:

- Delay in continuous assignments.
- Gate-level delays.
- Procedural delays.
- Min/typ/max delay values.
- Module path delays using the `specify` block.
- Timing checks inside the `specify` block.

A delay in a simulation model does not, by itself, determine the physical delay of the implemented hardware.

2 Timescale Directive

The ‘`timescale` compiler directive specifies the time unit and time precision used for delay interpretation.

2.1 Syntax

```
`timescale time_unit/time_precision
```

2.2 Example

```
`timescale 1ns/1ps

module timescale_example;
    reg a;
    wire y;

    assign #5 y = a;

    initial begin
        a = 0;
        #10 a = 1;
        #10 $finish;
    end
endmodule
```

Here:

- The time unit is 1 nanosecond.
- The time precision is 1 picosecond.
- The delay `#5` represents 5 ns.

The precision determines the resolution to which simulation delays are rounded.

3 Types of Delays in Verilog

Delay type	Description
Continuous assignment delay	Specifies delay in a continuous assignment.
Gate delay	Specifies delay for a built-in gate primitive.
Procedural delay	Uses delay controls such as `#5`.
Intra-assignment delay	Delays the update of a procedural assignment.
Module path delay	Specifies delays between module input and output ports using a `specify` block.
Timing checks	Checks timing relationships such as setup, hold, period, and pulse width.

4 Continuous Assignment Delay

A delay can be specified in a continuous assignment using the `#` symbol.

4.1 Syntax

```
assign #delay output_signal = expression;
```

4.2 Example

```
`timescale 1ns/1ps

module assignment_delay (
    input a,
    input b,
    output y
);

    assign #5 y = a & b;

endmodule
```

The output is scheduled to reflect the AND expression after the specified delay.

This type of delay is commonly used in simulation models.

5 Gate-Level Delay

Verilog gate primitives support delay specifications.

5.1 Example: AND Gate

```
`timescale 1ns/1ps

module gate_delay (
    input a,
    input b,
    output y
);

    and #(5) g1 (y, a, b);

endmodule
```

The gate primitive has a delay of 5 ns.

5.2 Separate Rise, Fall, and Turn-Off Delays

A gate can have different delays for rising, falling, and transition-to-high-impedance behavior.

```
`timescale 1ns/1ps

module tristate_gate_delay (
    input a,
    input enable,
    output y
);

    bufif1 #(3, 5, 7) b1 (y, a, enable);

endmodule
```

The three delay values represent rise, fall, and turn-off delays, respectively.

6 Procedural Delay

Procedural delays are used inside procedural blocks such as `initial` and `always`.

6.1 Example

```
`timescale 1ns/1ps

module procedural_delay;

    reg clk;

    initial begin
        clk = 0;
        #5 clk = 1;
        #5 clk = 0;
        #5 clk = 1;
        #5 $finish;
    end

endmodule
```

Each delay control suspends the execution of the current procedural block for the specified simulation time.

7 Intra-Assignment Delay

An intra-assignment delay is written between the assignment operator and the expression.

7.1 Example

```
`timescale 1ns/1ps

module intra_assignment_delay;

    reg a;
    reg q;

    initial begin
        a = 0;
        q = 0;

        #2 a = 1;
        q = #5 a;

        #10 $finish;
    end

endmodule
```

For a procedural assignment such as `q = #5 a`, the right-hand-side value is evaluated when the statement is reached, and the assignment to `q` occurs after the delay.

This differs from a delay control written before the statement.

8 Min, Typical, and Max Delays

Verilog supports min/typ/max delay values.

8.1 Syntax

```
assign #(minimum:typical:maximum) y = a;
```

8.2 Example

```
`timescale 1ns/1ps

module min_typ_max_example (
    input a,
    output y
);

    assign #(2:3:5) y = a;

endmodule
```

The values represent:

- Minimum delay: 2 ns.
- Typical delay: 3 ns.
- Maximum delay: 5 ns.

The simulator can select the minimum, typical, or maximum value according to its supported delay-selection settings.

9 Specify Block

A `specify` block describes module path delays and timing checks.

It is written inside a module, between the module's declarations and its `endmodule` statement.

9.1 General Syntax

```
module module_name (
    input a,
    output y
);

    specify

        specparam TPD = 3;

        (a => y) = TPD;

    endspecify

endmodule
```

Important elements include:

- `specify`: begins the specify block.
- `specparam`: declares a parameter for specify timing values.
- `(a => y)`: specifies a path from input `a` to output `y`.
- `endspecify`: ends the specify block.

Specify path delays are primarily intended for timing simulation. Tool support and simulation options determine whether these delays are applied.

10 Parallel and Full Module Paths

There are two commonly used module path descriptions.

10.1 Parallel Path

The parallel path operator is `=>`.

```
specify
    (a => y) = 3;
endspecify
```

A parallel path associates corresponding source and destination bits. It is commonly used for scalar paths and matching vector bits.

10.2 Full Path

The full path operator is *>.

```
specify
    (a *> y) = 3;
endspecify
```

A full path describes paths from each source bit to each destination bit for the declared source and destination expressions.

For scalar signals, the parallel and full path descriptions identify the same single source-to-destination connection.

11 Complete Specify Block Example

The following module models a simple inverter with rise, fall, and turn-off delay values.

```
`timescale 1ns/1ps

module inverter_timing (
    input a,
    output y
);

    not g1 (y, a);

    specify

        specparam
            T_RISE = 2,
            T_FALL = 3,
            T_TURN_OFF = 4;

        (a => y) = (T_RISE, T_FALL);

    endspecify

endmodule
```

The specify block defines separate rising and falling path delays.

The T_TURN_OFF parameter is included to illustrate a possible turn-off timing parameter, but it is not used by the path declaration above.

A turn-off delay is relevant to paths whose destination can transition to the high-impedance state, such as suitable tri-state outputs.

12 Three-Value Path Delays

A path delay can specify rise, fall, and turn-off delays.

12.1 Example

```
`timescale 1ns/1ps

module tristate_timing (
    input a,
    input en,
    output y
);

    bufif1 b1 (y, a, en);

    specify

        specparam
            T_RISE = 2,
            T_FALL = 3,
            T_OFF = 4;

        (a => y) = (T_RISE, T_FALL, T_OFF);

    endspecify

endmodule
```

The three values specify the rise, fall, and turn-off delays for the path.

The exact use of turn-off delays depends on the legal path declaration, output behavior, and simulator implementation.

13 Edge-Sensitive Module Paths

An edge-sensitive path associates a specified source transition with a destination path.

13.1 Example

```
`timescale 1ns/1ps

module edge_sensitive_path (
    input clk,
    input d,
    output q
);

    reg q_reg;

```

```
    always @(posedge clk)
        q_reg <= d;

    assign q = q_reg;

    specify

        specparam T_CLK_Q = 3;

        (posedge clk => (q +: 1'b1)) = T_CLK_Q;

    endspecify

endmodule
```

Edge-sensitive paths are useful for describing timing associated with clock transitions.

The destination polarity and edge association must accurately represent the intended timing behavior. For sequential cell timing, library models generally provide more complete timing specifications.

14 Conditional Specify Paths

A conditional path applies only when a specified condition is true.

The `if` construct can be used to describe conditional path delays.

14.1 Example

```
`timescale 1ns/1ps
module conditional_path (
    input a,
    input en,
    output y
);
    assign y = en ? a : 1'b0;

    specify
        specparam TPD = 3;
        if (en)
            (a => y) = TPD;
    endspecify
endmodule
```

The path delay applies when the condition `en` is true.

Conditional path descriptions are useful for circuits whose timing depends on the operating mode or control signals.

15 Specify Block Parameters

The `specparam` keyword declares timing parameters inside a specify block.

15.1 Example

```
`timescale 1ns/1ps

module specparam_example (
    input a,
    input b,
    output y
);

    assign y = a & b;

    specify

        specparam
            T_A_Y = 2,
            T_B_Y = 4;

        (a => y) = T_A_Y;
        (b => y) = T_B_Y;

    endspecify

endmodule
```

The two input-to-output paths have different specified delays.

A specify path delay models timing behavior for simulation; it does not insert a physical delay element into synthesized hardware.

16 Timing Checks in Specify Blocks

Specify blocks can also contain timing checks. These checks verify timing relationships between signals during timing simulation.

Common timing checks include:

Timing check	Purpose
`$setup`	Checks setup time before a reference event.
`$hold`	Checks hold time after a reference event.
`$setuphold`	Checks setup and hold requirements together.
`$recovery`	Checks recovery time for an asynchronous control signal.
`$removal`	Checks removal time for an asynchronous control signal.
`$period`	Checks the interval between reference events.
`$width`	Checks the width of a signal pulse.
`$skew`	Checks timing separation between events.

17 Example: Setup and Hold Timing Checks

The following example illustrates setup and hold checks for a data input relative to a clock edge.

```
`timescale 1ns/1ps

module setup_hold_example (
    input clk,
    input d,
    output reg q
);

    always @(posedge clk)
        q <= d;

    specify

        specparam
            T_SETUP = 2,
            T_HOLD = 1;

        $setup(d, posedge clk, T_SETUP);
        $hold(posedge clk, d, T_HOLD);

    endspecify

endmodule
```

The setup check verifies that the data signal does not change too close to the reference clock edge before it.

The hold check verifies that the data signal does not change too soon after the reference clock edge.

The exact timing-check argument conventions must follow Verilog timing-check syntax.

18 Example: Clock Period and Pulse Width

The `$period` and `$width` timing checks can verify clock timing.

```
`timescale 1ns/1ps

module clock_timing_checks (
    input clk
);

    specify

        specparam
            T_PERIOD = 10,
            T_WIDTH = 4;

        $period(posedge clk, T_PERIOD);
        $width(posedge clk, T_WIDTH);

    endspecify

endmodule
```

The period check verifies the minimum interval between consecutive positive edges.

The width check verifies the minimum duration of the pulse associated with the specified edge and its corresponding opposite edge.

Timing-check limits are evaluated by the simulator and can produce timing violations.

19 Specify Block vs. Assignment Delay

Feature	Assignment Delay	Specify Path Delay
Syntax	`assign #5 y = a;`	`(a => y) = 5;`
Purpose	Delays a continuous assignment.	Models a module input-to-output path.
Location	Module body.	Inside `specify`.
Timing checks	Not directly provided.	Can include setup, hold, and other checks.
Typical use	Simple delay models.	Cell and timing simulation models.

20 Important Notes About Specify Blocks

- A specify block begins with `specify` and ends with `endspecify`.
- Specify blocks are written inside module definitions.
- `specparam` is used to define timing parameters.
- Parallel paths use `=>`.
- Full paths use `*>`.
- Conditional paths can depend on module signals.
- Timing checks can detect setup, hold, period, and pulse-width violations.
- Simulator options may be needed to enable specify timing and timing checks.
- Specify delays do not guarantee physical hardware delays.
- Timing simulation may use standard cell library models with detailed timing data.

21 Common Errors and Best Practices

Common errors include:

- Forgetting `endspecify`.
- Writing path declarations outside the specify block.
- Using incorrect timing-check arguments.
- Confusing procedural delays with module path delays.
- Assuming specify delays automatically affect synthesis.
- Assuming all simulators enable timing checks by default.

Recommended practices include:

- Use consistent timescale settings.
- Define timing values using `specparam`.
- Verify path direction and polarity.
- Use the simulator documentation to enable timing simulation.
- Use appropriate cell timing models for detailed timing verification.
- Keep synthesizable RTL separate from simulation-oriented timing models when useful.

22 Summary

- Delay modeling represents timing behavior in Verilog simulation.
- Continuous assignments support delay specifications.
- Gate primitives support gate delays.
- Procedural delays control simulation execution within procedural blocks.
- Specify blocks describe module path delays and timing checks.
- `specparam` declares timing parameters.
- Timing checks detect violations of setup, hold, period, and pulse-width limits.

Key takeaway: Use ordinary delay controls for simple simulation delays and `specify` blocks for module path timing and timing checks. For accurate standard-cell timing verification, use suitable library models and simulator timing options.